



END TERM REVIEW REPORT

Multi Agent Reinforcement Learning for Flatland Environment

IIT Madras

Supervised by: Professor Chandrasekhar Lakshmi Narayanan

Submitted by

K.S MOHAN KUMAR (ME19B118)
Inter Disciplinary Dual Degree
Mechanical Engineering and Data Science
IIT Madras
Chennai

Contents

1	Introduction	1
2	Literature Survey	2
2.1	Scalable Rail Planning and Replanning with Soft Deadlines	2
2.1.1	Introduction	2
2.1.2	Problem Definition and Environment	2
2.1.3	MAPF and the Flatland Solution	2
2.1.4	Innovations in Flatland 3 Solution	3
2.1.5	Results and Conclusion	3
2.2	Scope Restriction for Scalable Real-Time Railway Rescheduling: An Exploratory Study	3
2.2.1	Introduction	3
2.2.2	Core Problem Concept	4
2.2.3	Scoping Strategies	4
2.2.4	Synthetic Environment and Instance Generation	5
2.2.5	Computational Results	5
2.2.6	Conclusion	6
2.3	Scalable Multi-Agent Model-Based Reinforcement Learning	6
2.3.1	Introduction	6
2.3.2	Methodology	6
2.3.3	Experiments	7
2.3.4	Results	7
2.3.5	Discussion and Conclusion	7
2.3.6	Conclusion	8
2.4	Standardized feature extraction from pairwise conflicts applied to the train rescheduling problem	8
2.4.1	Introduction and Motivation	8
2.4.2	Methodology	8
2.4.3	Key Contributions	8
2.4.4	Experimental Results	8
2.4.5	Conclusion and Future Directions	9
2.4.6	Significance	9
2.5	Winning Solution of the AICrowd SBB Flatland Challenge 2019-2020	9
2.5.1	Overview and Components	9
2.5.2	Path Generation Approach	10
2.5.3	Handling Malfunctions	10
2.5.4	Optimization and Performance	10
2.5.5	Unexplored Ideas	10
2.6	Flatland Competition 2020: MAPF and MARL for Efficient Train Co- ordination on a Grid World	10

2.6.1	Operations Research (OR) Solutions	11
2.6.2	An_Old_Driver	11
2.6.3	Reinforcement Learning (RL) Solutions	11
2.6.4	JBR_HSE	11
2.6.5	Netcetera	11
2.6.6	MARMot-Lab-NUS	11
2.6.7	Comparison and Conclusion	11
2.7	Symmetry Breaking for k-Robust Multi-Agent Path Finding	12
2.7.1	Foundational Concepts	12
2.7.2	Advanced Techniques for k-Robust Planning	13
2.7.3	Conclusion	13
2.8	Flatland-RL : Multi-Agent Reinforcement Learning on Trains	13
2.8.1	Introduction	13
2.8.2	Key Contributions and Findings	13
2.8.3	Conclusion	14
2.9	MABL: Bi-Level Latent-Variable World Model for Sample-Efficient Multi-Agent Reinforcement Learning	14
2.9.1	Introduction and Framework	15
2.9.2	Experimental Evaluation and Results	15
2.9.3	Conclusion	15
3	Introduction of MAPF	16
3.1	Overview of MAPF	16
3.2	Key Challenges	16
3.3	Applications	16
3.4	Conflict-Based Search (CBS)	17
3.4.1	Description	17
3.4.2	Approach	17
3.4.3	Advantages	17
3.5	Prioritized Planning (PP)	17
3.5.1	Description	17
3.5.2	Approach	17
3.5.3	Advantages	17
3.6	A* and Variants	17
3.6.1	Description	17
3.6.2	Approach	18
3.6.3	Advantages	18
3.7	Decentralized Approaches	18
3.7.1	Description	18
3.7.2	Approach	18
3.7.3	Advantages	18
3.8	Swarm Intelligence Algorithms	18
3.8.1	Description	18
3.8.2	Approach	18
3.8.3	Advantages	18
4	SIPP and CBS	19
4.1	Introduction	19
4.2	Conflict-Based Search (CBS) with Replanning	21
4.3	Conflict-Based Search (CBS)	21

4.3.1	Definitions for CBS	21
4.3.2	High Level of CBS	22
4.3.3	Processing a Node in the CT	22
4.4	Simple Replan Mechanism Inspired by Partial Replanning	24
4.5	Large Neighbourhood Search (LNS)	24
4.6	randomLNS for MAPF	25
5	HeatMap Observations	26
5.1	Graph Observations	26
5.2	Railway Network Representation and Traffic Analysis	27
5.2.1	Path Representation	27
6	Results from Running PPO on custom environments	28
6.1	Configuration-1	28
6.2	Configuration-2	29
6.3	Configuration-3	31
7	Multi Agent Path Finding using Tree LSTMs	33
7.1	Feature Extraction	33
7.2	Neural Network Architecture	33
7.3	TreeLSTM	34
7.4	Results	35
8	Custom Notations Added to the base environment	38
8.1	Description	38
8.2	Code Snippet	38
8.3	Observations	39
9	Future Work	40
9.1	Utilizing Flatland Observations with Graph Neural Networks	40
9.2	Incorporating Attention Mechanisms into PPO (MAMBA) Algorithm	40
9.3	Explainability for Trajectories and State Importance	40
9.4	Framework Integration with Ray RLLib or CleanRL	41
9.5	Conclusion	41

Chapter 1

Introduction

In this report, we explore the application of RL techniques in the context of "Flatland," a simulated environment developed for training RL agents on railway networks. Specifically, we focus on the challenges posed by scheduling and routing trains within this dynamic and often unpredictable environment. By leveraging RL, we aim to develop strategies that enhance the efficiency, reliability, and safety of railway operations.

This project investigates various aspects of Flatland RL, including problem formulation, algorithm selection, experimental setup, and performance evaluation. Through a series of simulations and analyses, we seek to uncover insights into how RL can be effectively applied to real-world railway networks, potentially revolutionizing the way we manage and optimize train operations.

The aim of this project until now is to get a clear understanding of how the FLATLAND works in a fundamental level and acquire the knowledge on the core fundamentals of the environment code to modify it for our own test cases. In this project we have also explored embedding the tamil Nadu railway stations into the Flatland environment and get inferences on the same.

Chapter 2

Literature Survey

2.1 Scalable Rail Planning and Replanning with Soft Deadlines

[5]

2.1.1 Introduction

The Flatland 3 Challenge, held as part of the NeurIPS 2020 conference, addresses the problem of efficiently managing dense traffic on complex railway networks, incorporating real-world elements like varying train speeds and scheduling time windows. Organized by AICrowd in collaboration with major European railway companies, the challenge builds on previous editions by introducing more realistic constraints such as the earliest departure times and expected arrival times for trains, as well as different speed profiles for passenger and freight trains.

The core task involves navigating trains on a grid-based railway network from their start stations to target stations while avoiding collisions and adhering to scheduling constraints. The environment simulates malfunctions and evaluates performance based on the success rate of trains reaching their destinations on time and minimizing arrival delays.

2.1.2 Problem Definition and Environment

Flatland 3 represents the railway network as a grid map with various rail types determining train movements. Each train has a start cell, target cell, speed profile, earliest departure time, and expected arrival time. The challenge aims to maximize the number of trains reaching their target cells and minimize total arrival delays. Trains may suffer malfunctions, requiring dynamic replanning.

2.1.3 MAPF and the Flatland Solution

The Multi-Agent Path Finding (MAPF) problem is central to Flatland 3, where agents (trains) must find collision-free paths on a graph. The winning solution from the NeurIPS 2020 competition employed a MAPF-based approach with several advanced techniques:

- **Prioritized Planning (PP)**: Initial planning using Safe Interval Path Finding (SIPP) to generate shortest paths for trains while avoiding collisions.

- **MAPF via Large Neighborhood Search (MAPF-LNS):** Repeatedly selects and replans subsets of agents to improve solution quality.
- **Minimal Communication Policy (MCP):** Handles train malfunctions by stopping some trains to avoid deadlocks.
- **Partial Replanning (PR):** Replans paths for agents affected by malfunctions.

2.1.4 Innovations in Flatland 3 Solution

The new solution for Flatland 3 extends the previous approach with additional improvements:

- **Slack-Based Priority:** A new priority ordering based on slack time (difference between expected arrival time and earliest departure time minus travel distance), prioritizing fast agents over slow ones.
- **Delay-Based Neighborhood Selection:** A strategy that focuses on agents causing delays and those affected by them, optimizing the total arrival delays.
- **Partial Replanning using LNS:** Enhances replanning by focusing on agents causing delays and optimizing the total arrival delays during execution, balancing between solution quality and runtime efficiency.

2.1.5 Results and Conclusion

The Flatland 3 solution was evaluated on 150 instances of varying difficulty, demonstrating its efficiency in planning and replanning paths for hundreds of trains within seconds. The winning solution achieved a high score of 135.47, solving 145 instances within the competition’s 2-hour limit, and outperforming both other MAPF-based solutions and reinforcement learning approaches.

In conclusion, the Flatland 3 Challenge highlighted the superiority of MAPF-based methods in managing complex railway networks. The winning approach effectively balances initial planning and dynamic replanning to handle real-world constraints, showcasing the practical applicability of MAPF in large-scale transportation problems.

2.2 Scope Restriction for Scalable Real-Time Railway Rescheduling: An Exploratory Study

[9]

2.2.1 Introduction

The paper addresses the complex challenge of real-time railway traffic management, focusing on efficient rescheduling of trains in response to disruptions. Traditional approaches often decompose the problem geographically, which can be computationally intensive and challenging to scale. The authors propose a novel approach termed the “core problem” concept, aiming to streamline and optimize the rescheduling process by narrowing down the problem scope dynamically based on disturbances.

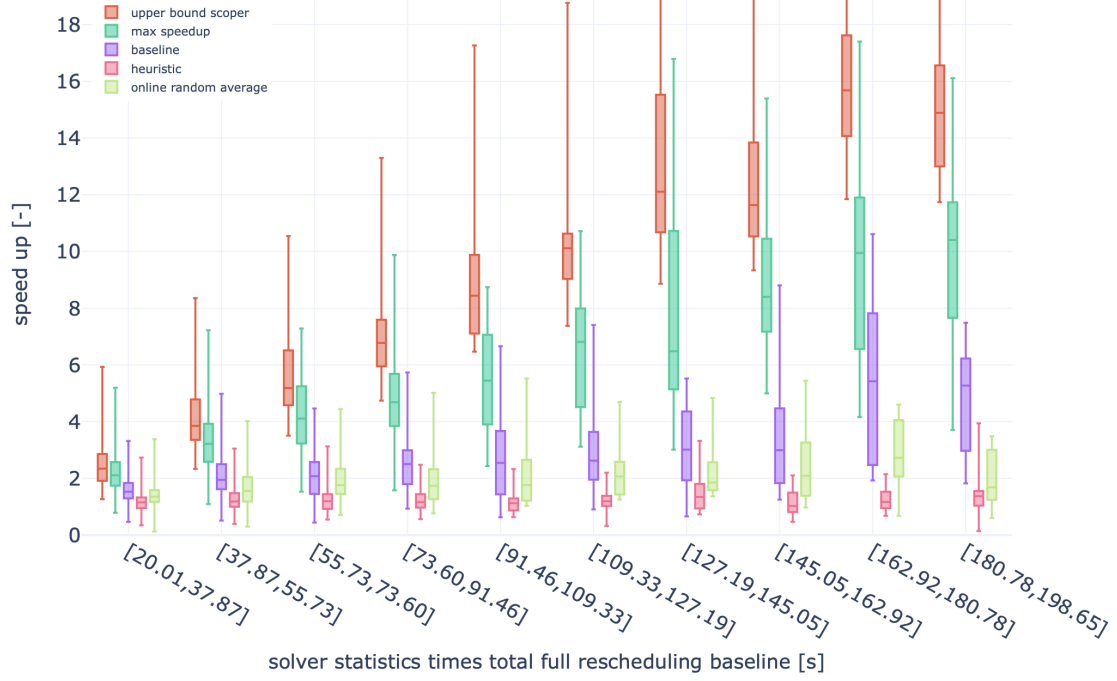


Figure 2.1: Speed-up, defined as the the factor between the computation time needed to solve the full scope and the restricted scope (per scoper), where a higher number means more improvement.

2.2.2 Core Problem Concept

Problem Context

Railway networks experience disruptions like malfunctions or delays, necessitating prompt rescheduling to minimize overall schedule disruptions and passenger inconveniences.

Innovation

The core problem concept departs from traditional spatial decomposition methods by defining the problem scope based on temporal and spatial parameters directly affected by disruptions. This approach aims to pinpoint specific trains and timeframes that require adjustment, thereby reducing computational complexity.

Methodology

The study utilizes Answer-Set Programming (ASP) within the Flatland simulation environment to model and solve the rescheduling problem. ASP proves advantageous for handling combinatorial optimization problems efficiently.

2.2.3 Scoping Strategies

The paper introduces several scoping strategies to define the core problem:

- **Max Speedup Scoper:** Restricts the problem scope to differences between the initial schedule and the online unrestricted solution. It serves as a non-trivial upper bound for speed-up.
- **Baseline Scoper:** Allows full routing flexibility for affected trains, providing insights into potential speed-up achievable on a per-train basis.
- **Heuristic Scoper:** Uses heuristic delay propagation to predict affected trains, serving as an initial attempt at an online scoper. It balances false positives and false negatives.
- **Random Scoper:** Uses randomly selected trains to validate solution quality against other scopers, acting as a baseline for comparison.

2.2.4 Synthetic Environment and Instance Generation

Infrastructure

The study employs the Flatland toolbox to generate 2D grid-based railway infrastructures with varying complexities. Cities and tracks are structured in a grid, with parallel tracks between cities to simulate realistic railway configurations.

Schedule

Train schedules are generated for each infrastructure configuration, defining origins, destinations, paths, and timings. Schedules are optimized to minimize overall train travel times within defined time constraints.

Malfunction Generation

Instances are created by simulating singular train malfunctions at specific times, aiming to test the responsiveness and effectiveness of the core problem approach under various disturbance scenarios.

2.2.5 Computational Results

Experimental Design

The study evaluates different scoping strategies across a diverse set of infrastructure and schedule configurations. Instances are categorized by computational difficulty based on the time required to solve the full re-scheduling problem.

Speed-Up Analysis

Results demonstrate significant computational speed-ups when using scoped problem definitions compared to the unrestricted full problem. The max speedup scoper achieves notable improvements, validating the efficacy of scoping strategies in reducing computational overhead.

Solution Quality

Solution quality metrics focus on minimizing temporal delays compared to the original schedule. The baseline and max speedup scopers consistently yield high-quality solutions, while the heuristic and random scopers serve as benchmarks for performance evaluation.

2.2.6 Conclusion

The paper concludes that accurately defining the core problem scope can lead to substantial computational speed-ups in railway traffic management. However, achieving an optimal scoping strategy requires balancing the identification of affected trains (avoiding false negatives) with maintaining computational efficiency (minimizing false positives). Future research directions include exploring machine learning approaches for scoper refinement and integrating real-time data for enhanced decision-making in railway operations.

2.3 Scalable Multi-Agent Model-Based Reinforcement Learning

[6]

2.3.1 Introduction

The paper introduces MAMBA (Model-based Approach for Multi-Agent environments), a novel method based on Model-Based Reinforcement Learning (MBRL) tailored for large-scale multi-agent systems. It addresses challenges such as sample inefficiency and scalability often encountered in Multi-Agent Reinforcement Learning (MARL). MAMBA leverages a world model to simulate agent interactions and employs decentralized communication through learned latent states.

2.3.2 Methodology

World Model and Communication Protocol

MAMBA utilizes a world model that predicts the next state based on the current state and actions of agents. This model is crucial for planning and decision-making without relying solely on trial-and-error exploration. Communication among agents is facilitated through a reward-agnostic protocol, where agents broadcast latent state representations derived from the world model rather than directly from rewards. This approach aims to provide unbiased information about the environment state rather than focusing solely on achieving specific goals.

Computation-Communication Trade-offs

To manage computational complexity, MAMBA implements efficient communication using attention mechanisms. Agents exchange information in a decentralized manner, focusing on local interactions to preserve scalability and reduce computational overhead. This approach contrasts with centralized methods that might struggle with large-scale environments.

2.3.3 Experiments

Environments

MAMBA is evaluated in two diverse environments: Flatland and StarCraft Multi-Agent Challenge (SMAC).

Flatland

Simulates train traffic on a rail network, where agents must navigate to destinations without collisions. The environment allows for the adjustment of traffic density and map size, posing challenges in coordination and scalability.

SMAC

Based on StarCraft II scenarios, requiring agents to engage in cooperative or competitive tasks with varying complexities. Challenges include micromanagement and terrain utilization, which demand effective coordination and communication strategies.

2.3.4 Results

SMAC Environments

MAMBA is compared against several state-of-the-art MFRL methods including MAPPO, COMA, QMIX, CommNet, and G2A across multiple SMAC scenarios. Even with limited training samples (100k), MAMBA achieves competitive or superior performance compared to these baselines. It demonstrates near-optimal performance in easier tasks and competitive results in more complex scenarios, underscoring its efficiency in learning from sparse data.

Flatland Environment

In Flatland, MAMBA competes against a modified version of the winning solution from the Flatland 2020 RL challenge. Results show that MAMBA outperforms this baseline, particularly in scenarios with lower agent counts and denser traffic, highlighting its ability to scale and generalize across varying environmental complexities.

2.3.5 Discussion and Conclusion

MAMBA’s performance in both Flatland and SMAC environments underscores the efficacy of MBRL in addressing the challenges of MARL. It achieves robust coordination and communication through learned latent states and a decentralized communication protocol, demonstrating superior sample efficiency compared to traditional MFRL methods. The paper concludes by discussing limitations, such as the increased computational complexity of learning a world model, and suggests avenues for future research to enhance MAMBA’s scalability and applicability in even larger and more complex multi-agent scenarios.

2.3.6 Conclusion

In conclusion, MAMBA represents a significant advancement in MARL, leveraging MBRL and decentralized communication to achieve efficient coordination and superior performance in challenging environments. Its ability to operate effectively with minimal training data sets it apart from traditional MFRL approaches, offering promising implications for real-world applications requiring scalable and adaptive multi-agent systems.

2.4 Standardized feature extraction from pairwise conflicts applied to the train rescheduling problem

[2]

2.4.1 Introduction and Motivation

Automated train rescheduling and traffic management are increasingly vital due to growing transportation needs. Current manual methods are suboptimal and heuristic-based. This paper explores the application of reinforcement learning (RL) to optimize train scheduling, drawing on recent advancements in RL for traffic management.

2.4.2 Methodology

The authors introduce a method to identify and resolve conflicts between pairs of trains in railway networks. They employ an analytical approach to model conflicts, providing optimal solutions in constant time. These conflict resolutions inform the construction of an observation space that captures essential decision-making information for RL algorithms.

2.4.3 Key Contributions

1. **Conflict Resolution:** Analytical solutions are developed for various conflict types (e.g., opposite and same directional conflicts) using the H-shaped Flatland model, ensuring efficient deadlock prevention.
2. **Observation Space Design:** Feature vectors are constructed based on conflict resolutions, focusing on actionable insights at decision points such as switches and intersections.
3. **Reinforcement Learning Application:** Constructed feature vectors serve as inputs to RL algorithms like Deep Q-Networks (DQN) and Linear Q-Networks (LQN), which learn optimal scheduling policies by maximizing reward signals (minimizing travel time and avoiding deadlocks).

2.4.4 Experimental Results

The proposed approach is evaluated using the Flatland Challenge benchmark, which simulates diverse railway configurations and agent scenarios. Preliminary findings

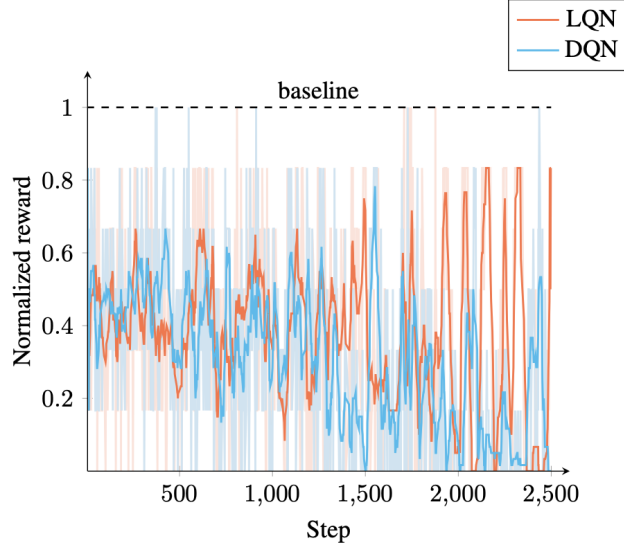


Figure 2.2: Evaluation of the training results of our proposed method based on the normalized reward values, in comparison with the baseline algorithm.

suggest that RL models trained on the proposed feature space perform competitively compared to a baseline A* algorithm, demonstrating potential for effective scheduling policy learning.

2.4.5 Conclusion and Future Directions

The study showcases the feasibility of standardized conflict-based feature extraction combined with RL for train rescheduling. Future research aims to refine feature spaces, explore different RL architectures, and validate the approach using historical railway data for enhanced real-world applicability.

2.4.6 Significance

The paper contributes to advancing automated train traffic management systems by integrating RL techniques with structured feature extraction from pairwise conflicts, addressing complex scheduling challenges in railway transportation efficiently.

2.5 Winning Solution of the AICrowd SBB Flatland Challenge 2019-2020

[1]

2.5.1 Overview and Components

The paper presents the winning solution that achieved a 99% success rate in routing agents to their destinations within the allotted time steps. The solution comprises two major components:

1. **Path (Re-)Generation:** Utilizes a time-expanded graph to generate paths for each agent. It employs permutations of agents based on their speeds and uses a shortest-path algorithm to compute non-interfering paths.
2. **Updating Paths after Malfunction:** Adjusts paths dynamically when malfunctions occur to maintain agent order and avoid deadlocks. It employs a precedence graph and a BFS-based algorithm to delay impacted agent visits while preserving visit orders.

2.5.2 Path Generation Approach

- Agents are initially assigned default paths outside the environment.
- Iteratively improves path assignments using random permutations and heuristic algorithms.
- A* search is employed in a time-expanded graph to compute paths, considering agent speeds and existing paths.

2.5.3 Handling Malfunctions

- Uses BFS to adjust paths post-malfunction, ensuring path validity and order preservation.
- Implements strategies to prevent deadlocks by managing agent movement overlaps and ensuring sequential cell visits.

2.5.4 Optimization and Performance

- Scores paths based on agents reaching destinations and time-related penalties.
- Balances path quality with computational efficiency, managing path re-generation frequency based on performance metrics.

2.5.5 Unexplored Ideas

- Discusses potential enhancements such as advanced deadlock avoidance strategies, multi-agent path optimization, and simulation-based parameter tuning.
- Highlights challenges and trade-offs in implementing these ideas within computational limits.

In conclusion, Andreica’s winning solution integrates path generation, dynamic path adjustment post-malfunction, and strategic deadlock prevention to achieve high success rates in the Flatland Challenge. The approach demonstrates effective use of graph-based algorithms, heuristic optimizations, and strategic simulations to manage complex multi-agent routing scenarios efficiently.

2.6 Flatland Competition 2020: MAPF and MARL for Efficient Train Coordination on a Grid World

[7]

2.6.1 Operations Research (OR) Solutions

2.6.2 An_Old_Driver

- **Method:** Prioritized Planning (PP) with A* and Large Neighborhood Search (LNS).
- **Key Features:** PP prioritized trains by planning shortest paths; LNS iteratively optimized paths.
- **Result:** Solved 363 instances with high completion rate and competitive scores.

2.6.3 Reinforcement Learning (RL) Solutions

2.6.4 JBR_HSE

- **Method:** Proximal Policy Optimization (PPO) with between-agent communication.
- **Key Features:** Agents used actor-critic models; communication enhanced coordination.
- **Result:** Achieved high score and competitive completion rate.

2.6.5 Netcetera

- **Method:** Centralized training with Ape-X model; agent prioritization using conflict graph.
- **Key Features:** Curriculum learning applied to train agents in complex environments.
- **Result:** Managed 88.1% completion with competitive score.

2.6.6 MARMot-Lab-NUS

- **Method:** Decentralized RL inspired by traffic light mechanisms for cluster management.
- **Key Features:** Used A3C algorithm with state masking; implemented traffic light policies.
- **Result:** Successfully handled 67% of agents with innovative traffic management.

2.6.7 Comparison and Conclusion

- **Performance:** OR solutions tackled more environments with high completion rates and competitive scores, leveraging centralized planning.
- **Advances in RL:** RL solutions showed promise with innovations in communication and traffic management, improving adaptability.

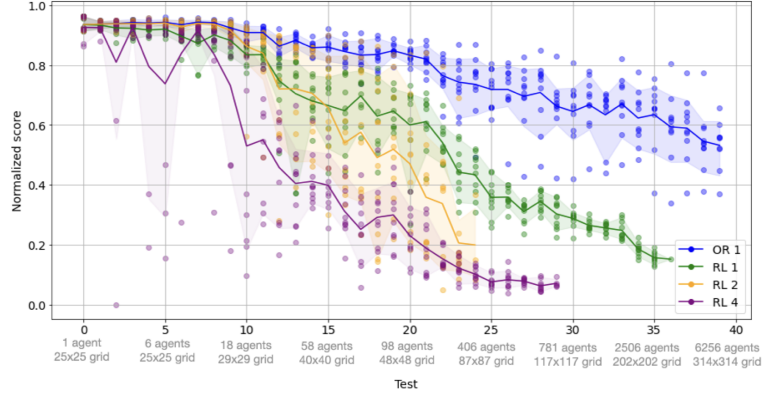


Figure 2.3: Ratio of agents that reached their targets per environment. The dots indicate the rate of agents that completed a given environment. The lines represent the mean of the 10 completion rates per test (x-axis) and the filled area the 0.1-quantile.

- **Challenges:** Scalability and coordination under dynamic conditions remain challenges for both OR and RL approaches.
- **Future Directions:** Further research needed to enhance RL methods with advanced observations and better coordination mechanisms.

2.7 Symmetry Breaking for k-Robust Multi-Agent Path Finding

[4]

The paper introduces advanced techniques to enhance Conflict-Based Search (CBS) algorithms for handling k-robust planning in Multi-Agent Pathfinding (MAPF) scenarios.

2.7.1 Foundational Concepts

- **CBS Framework:** CBS is used as the core algorithm for resolving conflicts between agents by branching on conflicting paths.
- **Rectangle Reasoning:** Techniques for handling conflicts in rectangular regions defined by agents' paths, classified into cardinal, semi-cardinal, and non-cardinal conflicts.
- **Conflict Detection:** Methods for detecting rectangle conflicts based on agent paths entering and leaving vertexes, considering different moving directions and barrier constraints.
- **Conflict Resolution:** Strategies such as k-MDDs (Multi-Valued Decision Diagrams) and branching methods (two-way vs. four-way) to efficiently resolve conflicts.

2.7.2 Advanced Techniques for k-Robust Planning

- **k-Robust Corridor Reasoning:** Extension of corridor symmetry-breaking techniques to handle k-delay conflicts occurring within corridors.
- **k-Robust Target Reasoning:** Addressing conflicts at agent goals by introducing constraints to prevent agents from conflicting at target locations due to k-delay issues.
- **Experimental Validation:** Validation of proposed algorithms through experiments across various scenarios including grid games, warehouse systems, and railway networks.
- **Performance Evaluation:** Comparison of enhanced CBS variants (K-CBSH, K-CBSH-RM, etc.) against baseline CBS, demonstrating improvements in success rates and conflict resolution efficiency as k (delay tolerance) increases.

2.7.3 Conclusion

The paper emphasizes the significance of symmetry resolution methods in generating k-robust plans for applications requiring robustness and safety in multi-agent environments, such as warehouse robotics and railway scheduling. It highlights the effectiveness of k-robust rectangle reasoning in reducing conflicts and enhancing the performance of MAPF algorithms. Future work is proposed to further optimize these methods and apply them to more complex real-world scenarios.

2.8 Flatland-RL : Multi-Agent Reinforcement Learning on Trains

[8]

2.8.1 Introduction

The paper on Flatland, a simulated railway environment, explores the application of Operations Research (OR) and Reinforcement Learning (RL) techniques to solve complex Virtual Rail Scheduling Problems (VRSP). It addresses challenges such as efficient navigation, path conflict resolution, and sparse, delayed rewards typical in such environments. The study leverages imitation learning (IL) alongside RL to integrate OR strategies into learning algorithms, aiming to generalize solutions beyond demonstrated scenarios.

2.8.2 Key Contributions and Findings

Imitation Learning Approaches

- **Pure Imitation Learning:** Utilizes stored expert experiences or mimics expert actions online. Achieved notable success with completion rates up to 86% when supplemented with actual environment data and a DQfD-inspired loss function.
- **Mixed Learning:** Combines offline (expert) and online (environment) experiences, showing improved completion rates over pure RL approaches.

Reinforcement Learning Strategies

- **Performance:** RL agents achieved completion rates ranging from 71% to 86% on test environments, demonstrating their ability to generalize but falling short of OR-based methods.
- **Algorithm Comparison:** Ape-X and PPO variants with different observations (global density, tree observations) showed competitive performance, with no significant sample efficiency differences observed.

Comparison with Operations Research (OR)

- **Superiority of OR Methods:** OR techniques consistently outperformed RL approaches on benchmark leaderboards, highlighting their strength in tackling scheduling and coordination challenges in railway systems.
- **RL Potential:** Despite OR dominance, RL showed promise, especially in scenarios where generalization from training to test environments was critical.

Limitations and Future Directions

- **Coordination Challenges:** RL agents struggled with inter-agent coordination, leading to deadlock situations, suggesting potential improvements with communication mechanisms.
- **Real-world Applicability:** Simplified models in Flatland mirror real-world railway planning principles, aiding in developing prototypes for traffic management systems.

2.8.3 Conclusion

The study underscores that while RL in Flatland can effectively address VRSP, OR methods currently offer superior performance due to their specialized nature in railway planning. The findings advocate for further exploration to enhance RL’s coordination capabilities and adaptability to real-world railway networks, aiming to bridge the gap between theoretical benchmarks and practical applications in railway operations.

This comprehensive approach positions the Flatland environment as a crucial testbed for advancing both OR and RL methodologies in complex scheduling and coordination problems, with implications extending to real-world railway planning and management systems.

2.9 MABL: Bi-Level Latent-Variable World Model for Sample-Efficient Multi-Agent Reinforcement Learning

[11]

The paper introduces Multi-Agent Bi-Level (MABL) models, a novel approach in Multi-Agent Reinforcement Learning (MARL) that enhances sample efficiency through bi-level latent-variable structures. MABL addresses challenges in capturing complex interactions among agents by integrating local agent-specific information with global context using latent-variable models.

2.9.1 Introduction and Framework

The paper starts by highlighting the limitations of traditional MARL algorithms in handling multi-agent interactions efficiently. It proposes the use of bi-level latent-variable models to improve policy learning and decision-making in complex environments. The key components of MABL include:

- **Problem Statement:** Addressing sample inefficiency in MARL, particularly in environments with extensive agent interactions.
- **Bi-Level Structure:** Introducing a hierarchical model where each agent’s latent state is conditioned on both local features and a global latent state, facilitating information flow and capturing dependencies across agents.
- **Model Architecture:** Describing the architecture involving:
 - Representation Model: Learning latent representations encoding both local and global context.
 - Transition Dynamics Model: Predicting future states based on current states and actions.
 - Auxiliary Components: Including reward predictors, termination predictors, and action decoders to enhance learning.

2.9.2 Experimental Evaluation and Results

The paper evaluates MABL through experiments on benchmarks such as SMAC, Flatland, and MAMuJoCo, comparing its performance in sample efficiency against state-of-the-art MARL algorithms. Key findings include:

- **Experimental Setup:** Conducting experiments under low data regimes to assess MABL’s performance against CTDE and CTCE baselines.
- **Performance Evaluation:** Demonstrating superior sample efficiency of MABL across benchmarks. It outperforms CTDE in win-rates and rewards, and competes with CTCE despite lacking global information at execution.
- **Ablation Study:** Investigating the impact of bi-level structure on performance gains, confirming its role in enhancing representation learning and policy effectiveness in MARL tasks.

2.9.3 Conclusion

In conclusion, "Learning Multi-Agent Behavior with Bi-Level Latent-Variable Models" introduces MABL as a promising approach to enhance scalability and sample efficiency in MARL. By leveraging bi-level latent-variable models, MABL effectively captures both local and global dependencies among agents, thereby improving decision-making in complex environments. Experimental results validate MABL’s superiority over existing approaches, indicating its potential to advance the field of MARL research.

Chapter 3

Introduction of MAPF

3.1 Overview of MAPF

Multi-Agent Path Finding (MAPF) is a fundamental problem in artificial intelligence and robotics, where the objective is to plan paths for multiple agents in a common environment. Each agent has a unique starting position and goal destination, and they must navigate through the environment while avoiding collisions with other agents and obstacles.

3.2 Key Challenges

The primary challenges in MAPF include:

- **Collision Avoidance:** Ensuring that agents do not collide with each other while moving towards their goals.
- **Path Optimality:** Finding paths that minimize total travel time or other metrics while considering the dynamic nature of the environment.
- **Scalability:** Efficiently handling a large number of agents and complex environments.
- **Dynamic Environments:** Adapting plans in real-time to changes such as new obstacles or updated goals.
- **Decentralization:** Coordinating agents without relying on a centralized controller, which is crucial for scalability and robustness.

3.3 Applications

MAPF has applications in various domains, including:

- **Robotics:** Autonomous multi-robot systems for tasks such as exploration, surveillance, and transportation.
- **Logistics and Transportation:** Planning routes for multiple vehicles or drones in urban environments.

- **Video Games:** NPC (non-player character) movement in games requiring realistic and coordinated behaviors.
- **Emergency Response:** Coordinating rescue robots or drones in disaster scenarios.

3.4 Conflict-Based Search (CBS)

3.4.1 Description

Conflict-Based Search (CBS) is a framework for MAPF that systematically searches for conflicts among agents and resolves them to find feasible paths for all agents.

3.4.2 Approach

CBS employs a two-level search strategy:

- **High-Level Search:** Constructs paths for each agent independently.
- **Low-Level Search:** Resolves conflicts by modifying paths of conflicting agents.

3.4.3 Advantages

- Effective for handling complex environments and dynamic changes.
- Incremental conflict resolution approach ensures feasibility of paths.

3.5 Prioritized Planning (PP)

3.5.1 Description

Prioritized Planning assigns priorities to agents and plans paths for higher priority agents first, assuming lower priority agents will accommodate them.

3.5.2 Approach

- Uses A* or other graph-based search algorithms to find paths.
- Considers constraints and paths of higher priority agents.

3.5.3 Advantages

- Straightforward implementation.
- Suitable for a wide range of MAPF scenarios.

3.6 A* and Variants

3.6.1 Description

A* and its variants adapt the single-agent pathfinding algorithm for MAPF by considering multiple agents, their positions, goals, and constraints.

3.6.2 Approach

- Variants like Conflict-Based A* or Cooperative A* modify heuristics and search strategies.
- Efficient in both centralized and decentralized MAPF approaches.

3.6.3 Advantages

- Efficient and effective in pathfinding for multiple agents.
- Can be adapted to handle dynamic environments.

3.7 Decentralized Approaches

3.7.1 Description

Decentralized approaches distribute planning and decision-making among agents, reducing the need for centralized coordination.

3.7.2 Approach

- Each agent plans its path independently.
- Considers positions and plans of other agents via communication protocols.

3.7.3 Advantages

- Scalable and robust in dynamic environments.
- Suitable for scenarios with limited communication or computational resources.

3.8 Swarm Intelligence Algorithms

3.8.1 Description

Algorithms inspired by swarm behavior in nature, adapted for MAPF to coordinate agents based on local interactions and emergent behaviors.

3.8.2 Approach

- Agents adjust paths based on local information and interactions.
- Aim to collectively optimize overall performance.

3.8.3 Advantages

- Effective in dynamic and emergent scenarios.
- Can handle complex environments with distributed decision-making.

Chapter 4

SIPP and CBS

4.1 Introduction

Whether autonomously driving a vehicle or performing household tasks like room cleaning, robots must navigate safely in the presence of moving objects such as people, pets, or cars. Predicting the trajectories of these dynamic obstacles and planning paths quickly are essential for robot operations in changing environments. Planning in dynamic environments requires adding a time dimension to the state-space, which increases the number of states to be searched and consequently prolongs planning times. Real-time response necessitates generating plans quickly to account for unpredictable obstacle trajectories.

For efficiency, many planners treat dynamic obstacles as static or use simplified trajectory predictions to restrict planning to safe configurations, avoiding collisions in the near future. However, these methods may result in suboptimal or incomplete solutions, bypassing feasible paths or failing in complex scenarios.

The chosen approach to tackle sequential multi agent path finding is Safe Interval Path Planning, which is an extension on A* in which the notion of safe intervals is implemented. Safe intervals are contiguous periods of time where a given location on a grid does not contain an obstacle. The key benefit to using SIPP is that when these safe intervals are used to index the agent states instead of timesteps, the search space can be significantly reduced. Additionally, SIPP nodes implicitly wait in place, thus removing the need to model “wait” actions explicitly like in TXA* and reducing the search space further. Figure 2 below compares the runtimes of the implemented TXA* and SIPP across levels 0 to 5 and demonstrates that SIPP plans agents extremely quickly compared to TXA*. The use of the safe interval concept and the mechanic of moving on as early as possible allows Safe Interval Path Planning to have a strong advantage over other time-based A* alternatives. This is because the full dimension of time reduces to contiguous time intervals where an agent is not in collision. The number of these safe intervals is much smaller than the number of underlying timesteps. For example, a small timescale of $0 \dots 10$ with an obstacle at timestep 5 is reduced to just two intervals, $[0,4]$ and $[6,10]$. Thus, we only search over these two intervals, instead of all of $0 \dots 10$ timesteps.

Algorithm 1: A* with Safe Intervals

Algorithm 1 A* with safe intervals

```
1:  $g(s_{\text{start}}) = 0$ , OPEN =  $\emptyset$ 
2: Insert  $s_{\text{start}}$  into OPEN with  $f(s_{\text{start}}) = h(s_{\text{start}})$ 
3: while  $s_{\text{goal}}$  is not expanded do
4:   Remove  $s$  with the smallest  $f$ -value from OPEN
5:   successors = getSuccessors( $s$ )
6:   for each  $s'$  in successors do
7:     if  $s'$  was not visited before then
8:        $f(s') = g(s') = \infty$ 
9:     end if
10:    if  $g(s') > g(s) + c(s, s')$  then
11:       $g(s') = g(s) + c(s, s')$ 
12:      updateTime( $s'$ )
13:       $f(s') = g(s') + h(s')$ 
14:      Insert  $s'$  into OPEN with  $f(s')$ 
15:    end if
16:  end for
17: end while
```

Algorithm 2: getSuccessors(s)

Algorithm 2 getSuccessors(s)

```
1: successors =  $\emptyset$ 
2: for each action in valid transitions do
3:   cfg = configuration of action applied to  $s$ 
4:   m_time = time to execute action
5:   start_t = time( $s$ ) + m_time
6:   end_t = time( $s$ ) + endTime(interval( $s$ ))
7:   for each safe interval in cfg do
8:     if startTime( $i$ ) > end_t or endTime(interval( $i$ )) < start_t then
9:       continue
10:    end if
11:     $t = \max(\text{start\_t}, \text{startTime}(i))$ 
12:    if edge or vertex collision with existing paths then
13:      continue
14:    end if
15:    if  $t > \min(\text{end\_t}, \text{endTime}(\text{interval}(i)))$  then
16:      continue
17:    end if
18:     $g = t - \text{time}(s)$ 
19:     $s' = \text{state of cfg with interval } i \text{ and time } t$ 
20:    with time to act cost  $g$  applied
21:    Insert  $s'$  into successors
22:  end for
23: end for
24: return successors
```

4.2 Conflict-Based Search (CBS) with Replanning

4.3 Conflict-Based Search (CBS)

4.3.1 Definitions for CBS

The following definitions are used throughout the CBS algorithm:

- **Path:** Refers to a single-agent path.
- **Solution:** Denotes a set of k paths for k agents.
- **Constraint:** A tuple (i, v, t) where agent i is prohibited from occupying vertex v at time step t .
- **Consistent path:** A path that satisfies all its associated constraints.
- **Consistent solution:** A set of paths where each path is consistent with its agent's constraints.
- **Conflict:** A tuple (i, j, v, t) where agents i and j occupy vertex v at time step t .

Algorithm 3 CBS with Replanning

Require: Agents, Rail, Initial paths, Max timestep**Ensure:** Paths after replanning

```
1: function REPLAN(Agents, Rail, Current timestep, Existing paths, Max timestep, New malfunction)
2:   Paths_before_current_time  $\leftarrow$  Existing paths[:, :
   min(len(Existing paths), Current timestep)]
3:   Unfinished_ids  $\leftarrow$  []
4:   for agent_id in range(len(Agents)) do
5:     if Agents[agent_id].position  $\neq$  None then
6:       Unfinished_ids.append(agent_id)
7:     end if
8:   end for
9:   Agents_re  $\leftarrow$  [Agents[id] for id in Unfinished_ids]
10:  Path_after_timespace  $\leftarrow$  get_replan_paths(Agents_re, Rail, Max timestep)
11:  for i, id in enumerate(Unfinished_ids) do
12:    Existing paths[id]  $\leftarrow$  Paths_before_current_time[id] +
    Path_after_timespace[i]
13:  end for
  return Existing paths
14: end function
```

```
Input: MAPF instance
1 Root.constraints = {}
2 Root.solution = find individual paths by the low level()
3 Root.cost = SIC(Root.solution)
4 insert Root to OPEN
5 while OPEN not empty do
6   P  $\leftarrow$  best node from OPEN // lowest solution cost
7   Validate the paths in P until a conflict occurs.
8   if P has no conflict then
9     return P.solution // P is good
10  C  $\leftarrow$  first conflict (a, a', v, t) in P
11  if shouldMerge(a, a') // Optional, MA-CBS only then
12    aij = merge(a, a', v, t)
13    Update P.constraints(external constraints)
14    Update P.solution by invoking low level(aij)
15    Update P.cost
16    if P.cost <  $\infty$  // A solution was found then
17      Insert P to OPEN
18    continue // go back to the while statement
19  foreach agent ai in C do
20    A  $\leftarrow$  new node
21    A.constraints  $\leftarrow$  P.constraints + (ai, v, t)
22    A.solution  $\leftarrow$  P.solution
23    Update A.solution by invoking low level(ai)
24    A.cost  $\leftarrow$  SIC(A.solution)
25    if A.cost <  $\infty$  // A solution was found then
26      Insert A to OPEN
```

4.3.2 High Level of CBS

4.3.3 Processing a Node in the CT

- **Low-level search:** Finds a path for an agent consistent with current constraints.
- **Validation:** Checks for conflicts among paths in the solution.
- **Conflict resolution:** Splits nodes and adds new constraints to resolve conflicts.[10]

Algorithm 4 High level of CBS

Require: Agents, Map, Constraints

Ensure: Optimal solution (set of paths)

```
1: function CBS_HIGH_LEVEL(Agents, Map, Constraints)
2:   Initialize Constraint Tree (CT) with root node containing  $\emptyset$ , empty solution,
   cost 0
3:   Insert root node into priority queue OPEN
4:   while OPEN is not empty do
5:     Extract node N from OPEN
6:     if N is a goal node (solution is valid) then
7:       return solution in N
8:     end if
9:     for each agent i do
10:      Perform low-level search to find a path consistent with N's constraints
      for agent i
11:      Validate paths against conflicts
12:      if no conflicts then
13:        Create child nodes by adding new constraints for agent i
14:        Insert children into OPEN
15:      end if
16:    end for
17:  end while
18: end function
```

4.4 Simple Replan Mechanism Inspired by Partial Replanning

Algorithm 5 `replan(agents, malfunction_data, timestep)`

```

1: malfunctioned_agents = agents that have malfunctioned according to malfunction_data
2: all_paths = {(path(a)) |  $\forall a \in \text{agents}$ }
3: paths_from_timestep = {slice path from  $t$  onwards |  $\forall p \in \text{all\_paths}$ }
4: malfunctioned_paths = { $p[t \dots]$  |  $\forall \text{malfunctioned\_agents}$ }
5: affected = {}
6: for each  $m\_path$  in malfunctioned_paths do
7:   agent_id = 0
8:   for each path in paths_from_timestep \ malfunctioned_paths do
9:     if path intersects  $m\_path$  and agent_id not in malfunctioned_agents then
10:      insert agent_id into affected
11:      agent_id+ = 1
12:    end if
13:  end for
14: end for
15: for each  $m\_agent$  in malfunctioned_agents do
16:   delayed_path = repeat  $(x, y)$  location  $\text{path}(m\_agent)@t$  for length of malfunction
17:   replace original with delayed_path in all_paths for  $m\_agent$ 
18: end for
19: for each agent in affected do
20:   replan = call Algorithm 1 with affected_agent and all_paths \
    path(agent) as collisions
21:   replace original with replan in all_paths for agent
22: end for
23: return all_paths

```

4.5 Large Neighbourhood Search (LNS)

Large Neighbourhood Search (LNS) is a metaheuristic approach used for solving combinatorial optimization problems by exploring large sets of diverse neighborhoods of the solution space. It involves iteratively destroying and repairing parts of the current solution to potentially find better solutions.

- **Diverse Exploration:** LNS explores a large number of diverse neighborhoods rather than exhaustively searching through all possible solutions.
- **Destroy and Repair:** It iteratively destroys parts of the current solution (destroy phase) and then repairs them using a heuristic or algorithm (repair phase).
- **Iterative Improvement:** LNS repeats the destroy and repair phases multiple times, evaluating the quality of each new solution and accepting it based on certain criteria.

LNS is particularly effective for complex optimization problems where finding an exact optimal solution is impractical due to computational constraints.

4.6 randomLNS for MAPF

Algorithm 6 describes the randomLNS algorithm for Multi-Agent Path Finding (MAPF) using Large Neighbourhood Search (LNS). It aims to improve the quality of paths generated for multiple agents by iteratively exploring diverse neighborhoods of agent configurations.

Algorithm 6 randomLNS(*agents*, *early_stop*=5)

$best_SIC = \sum_{a \in agents} (|path(a)| - |agents|);$ $best_penalty = \sum_{a \in agents} (\max(0, |path(a)| - deadline(a)));$ $best_score = best_SIC + best_penalty;$
 $best_paths = \{path(a) \mid a \in agents\};$ $count = 0;$ $iterations < max_iterations$ $count > early_stop$ $best_paths;$ $n \sim Uniform(1, |agents|);$
 $sampld_agents =$ randomly sample n agents; $new_paths = \{agents\} \setminus \{sampld_agents\};$ # destroy each agent a in $sampld_agents$ $path =$
call Algorithm 1 with a and new_paths as collisions; insert $path$ into $new_paths;$
 $new_score = score(new_paths);$ $new_score < best_score$ $best_score = new_score;$
 $best_paths = new_paths;$ $count = 0;$ $count = count + 1;$ $best_paths;$

This algorithm uses random sampling to select subsets of agents, destroys their paths, and then repairs them using a specified algorithm (here, Algorithm 1) to potentially find better solutions for MAPF.[3]

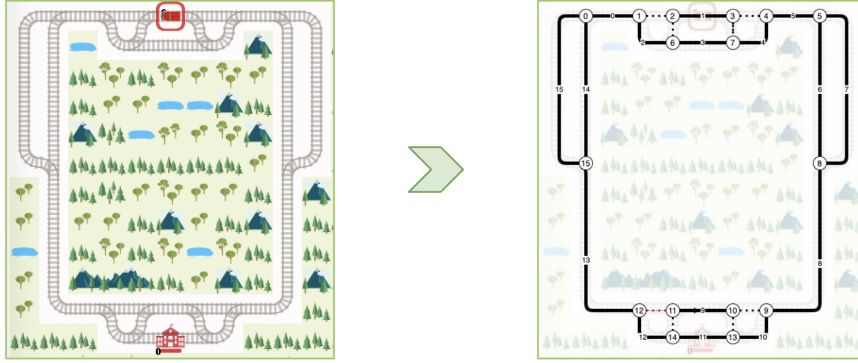
Chapter 5

HeatMap Observations

5.1 Graph Observations

In our investigation of navigation and conflict avoidance strategies for railway systems, we explore a dual approach involving high-level pathfinding algorithms and low-level deep reinforcement learning (DRL) methodologies. The high-level strategy utilizes shortest path algorithms to determine optimal routes, while the low-level approach focuses on real-time conflict avoidance using DRL.

The decision-making process at the high level is streamlined, reducing actions to 'go'



(0) and 'stop' (1), with stopping actions reserved for conflict resolution. Our observations, termed Graph Observations, leverage the railway network's graph structure. Here, nodes represent switches and edges denote paths between nodes, providing a global perspective on connectivity crucial for multi-agent scenarios.

To enhance conflict avoidance, we propose a observation format based on the agent's chosen path, represented as a sequential array of cells. This allows the agent to anticipate and mitigate potential conflicts by analyzing features along its predicted path depth.

Our findings suggest that integrating these high-level pathfinding and low-level DRL approaches not only optimizes navigation efficiency but also significantly enhances safety through proactive conflict management.

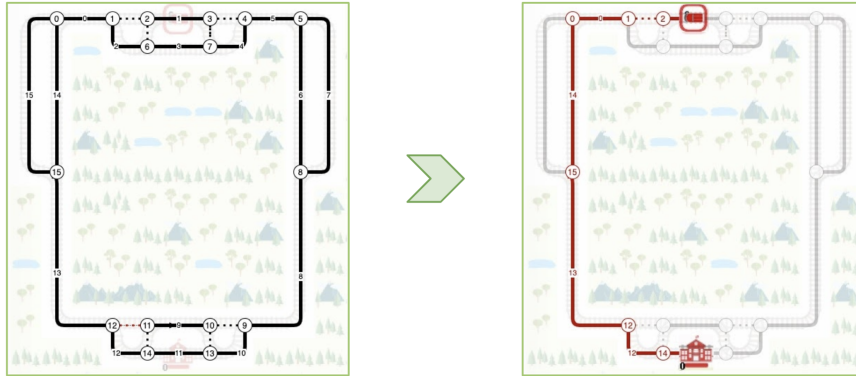
5.2 Railway Network Representation and Traffic Analysis

A directed graph representation of the railway network is constructed using BFS, where each node represents a switch and each edge represents a rail between two switches. The shortest path for each agent is computed on this graph.

5.2.1 Path Representation

Each computed path is transformed into a bitmap format for further analysis. In this representation:

- Time steps are represented as columns.
- Rails (edges between switches) are represented as rows.
- The direction of travel on each rail is indicated:
 - Positive (1) if the agent is traveling from the source node to the destination node of the rail.
 - Negative (-1) if the agent is traveling in the opposite direction.



Chapter 6

Results from Running PPO on custom environments

6.1 Configuration-1

Custom sparse environment with 3 agents and 3 cities with a grid size of 80x80.

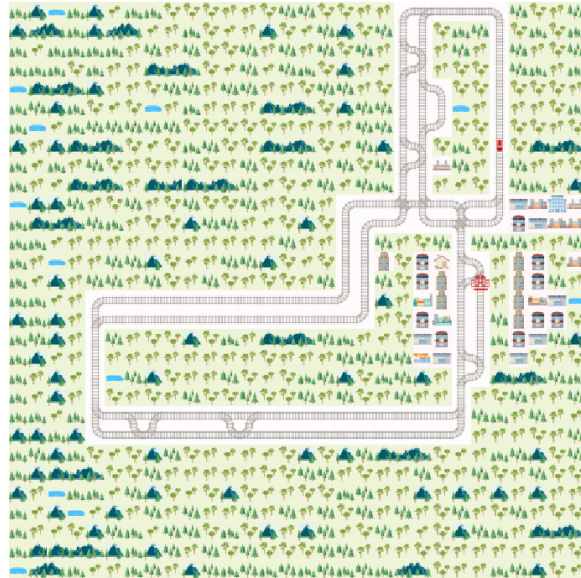
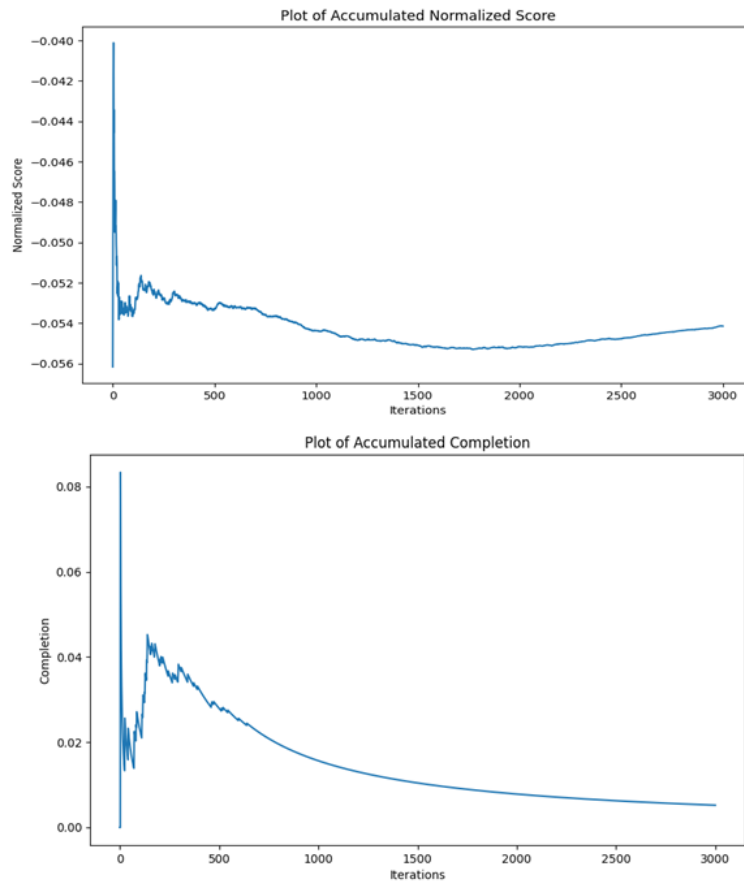


Figure 6.1: [Link to gif](#)



6.2 Configuration-2

30 agents, 20 cities in 125x125 grid

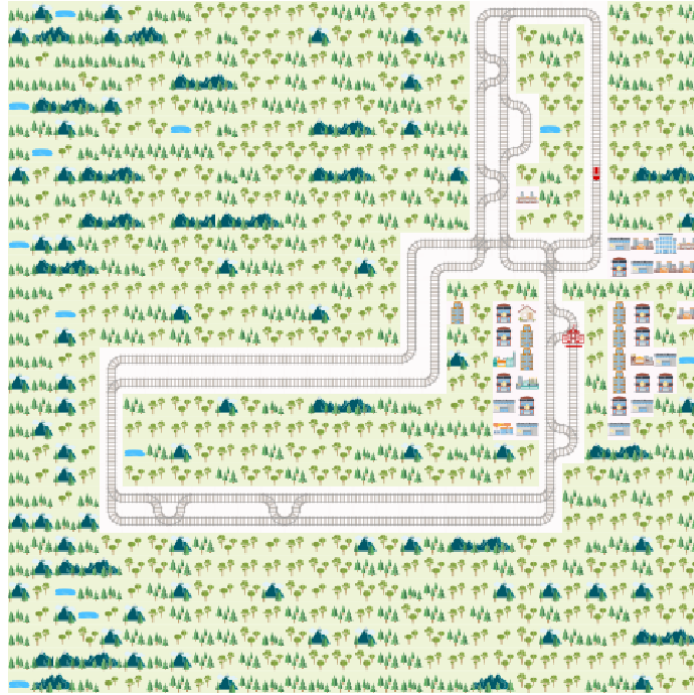
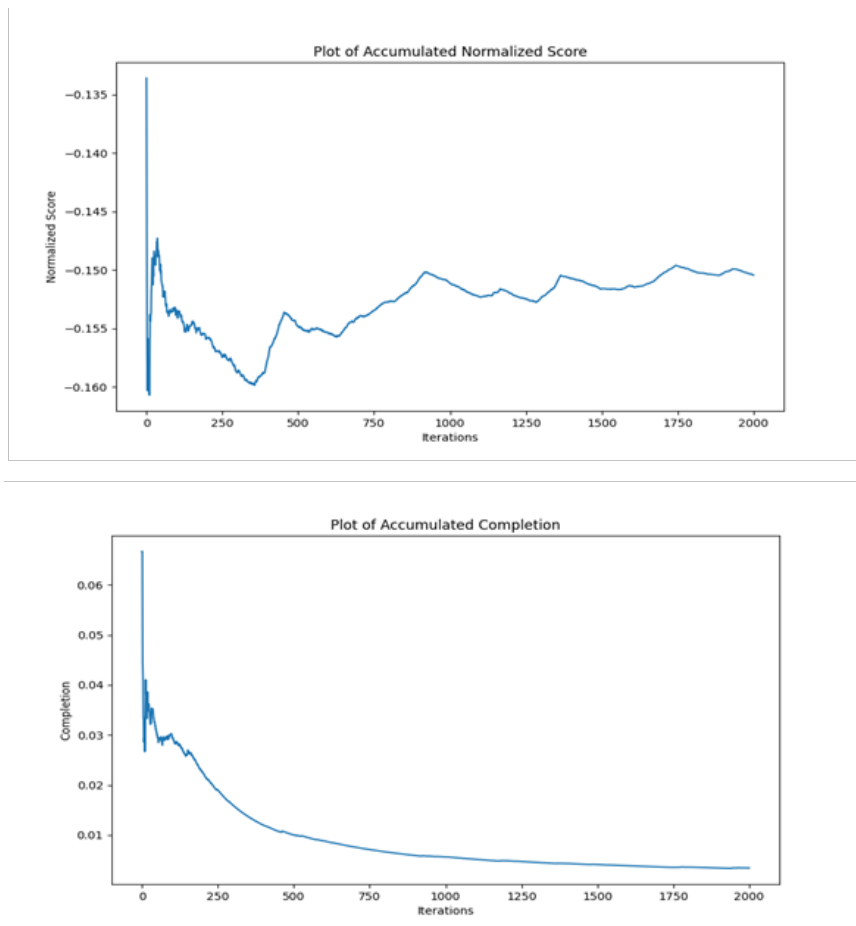


Figure 6.2: Link to gif



6.3 Configuration-3

Custom dense environment with 3 agents and 3 cities with a grid size of 80x80.

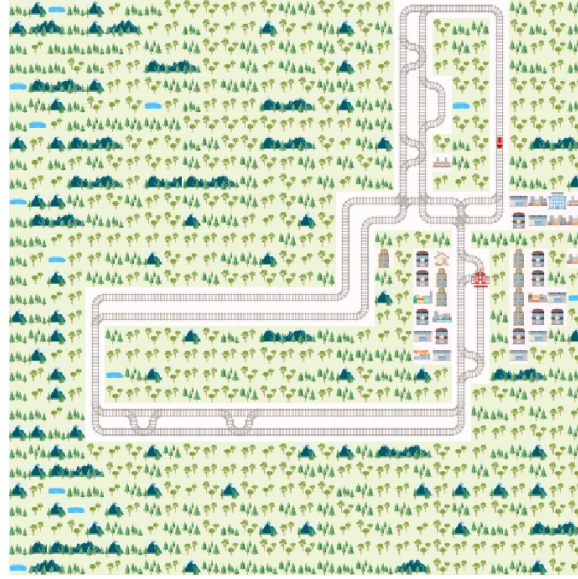
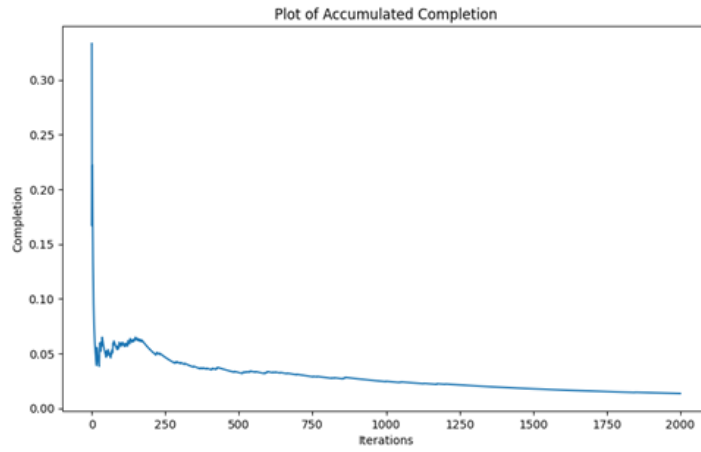
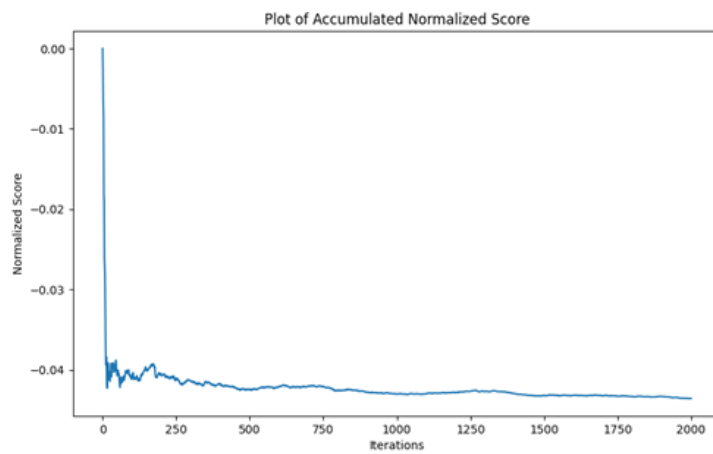


Figure 6.3: Link to gif



Chapter 7

Multi Agent Path Finding using Tree LSTMs

□

7.1 Feature Extraction

The features extracted consist of two parts: **Xattr** and **Xtree**.

Agent Attributes (Xattr) include attributes such as ID, earliest departure time, latest arrival time, current state, direction, and remaining time.

Tree Representation of Possible Future Paths (Xtree) encodes possible future paths as a spanning tree constructed via depth-limited BFS from the agent's current location. Each node in this tree represents a potential branch the agent may choose.

The spanning tree for each agent i is denoted as $T_i = (V_i, E_i)$, where each node $\nu \in V_i$ is associated with a vector x_ν containing various useful information about that node.

7.2 Neural Network Architecture

The neural network architecture processes **Xattr** using a 4-layer MLP and **Xtree** using a TreeLSTM (*Tree Long Short-Term Memory*), designed for handling tree-structured data.

$$\begin{aligned} H_{\text{attr}} &= \text{MLP}(X_{\text{attr}}) \\ H_{\text{tree}} &= \text{TreeLSTM}(X_{\text{tree}}) \\ H^{(0)} &= [H_{\text{attr}}, H_{\text{tree}}] \\ H^{(l)} &= \text{Self-Attention}(H^{(l-1)}), \quad l = 1, 2, 3 \\ A &= \text{MLP}(H^{(3)}) \\ V &= \text{MLP}(H^{(3)}) \end{aligned}$$

The outputs A and V represent action logits and estimated state-value respectively.

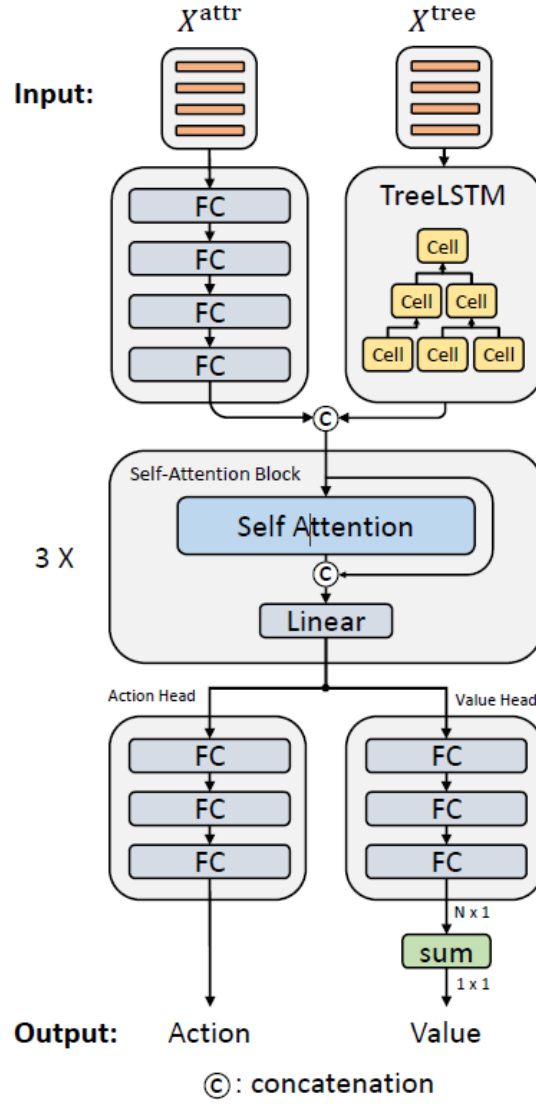


Figure 7.1: Overview of network architecture

7.3 TreeLSTM

TreeLSTM is a variant of LSTM tailored for tree-structured data. It processes each node considering its children's outputs:

$$(c_t, h_t) = \text{TreeLSTM-Cell}(x_t, \{(h_k, c_k) \mid k \in \text{Child}(t)\})$$

In this network, Child-sum TreeLSTM aggregation is used, where the states of children are summed to form the input to each TreeLSTM cell.

7.4 Results

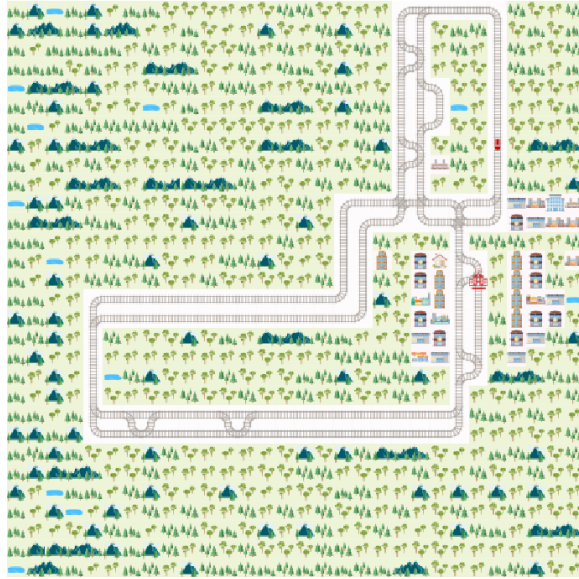


Figure 7.2: [Link to video](#)

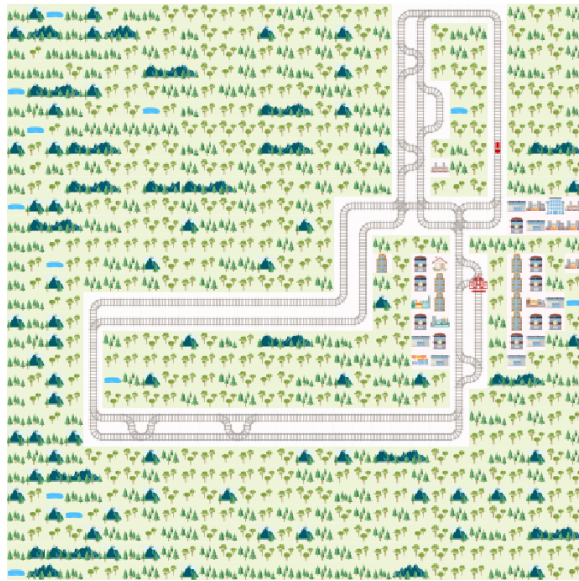


Figure 7.3: [Link to video](#)

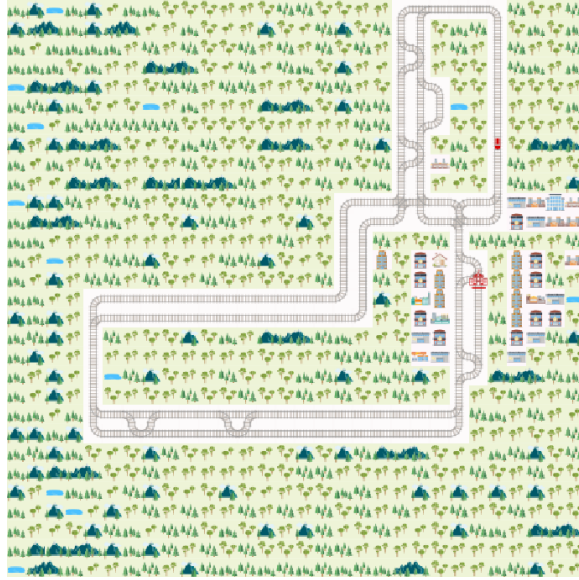


Figure 7.4: [Link to video](#)

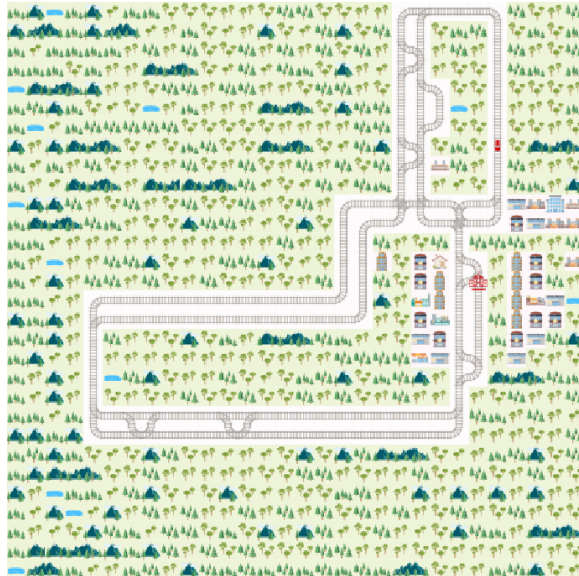


Figure 7.5: [Link to video](#)

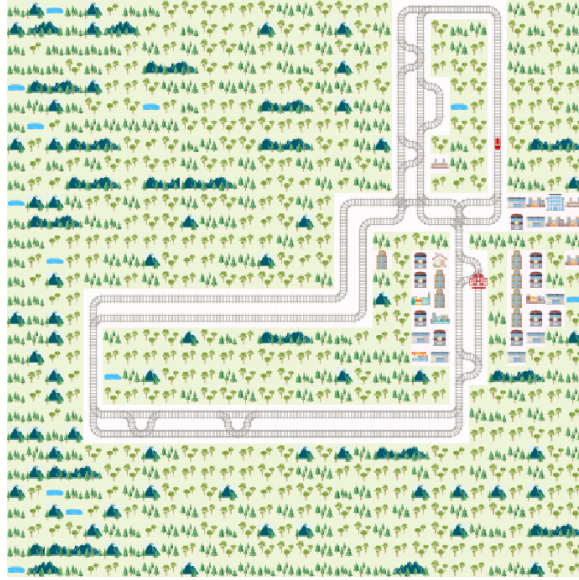


Figure 7.6: [Link to video](#)

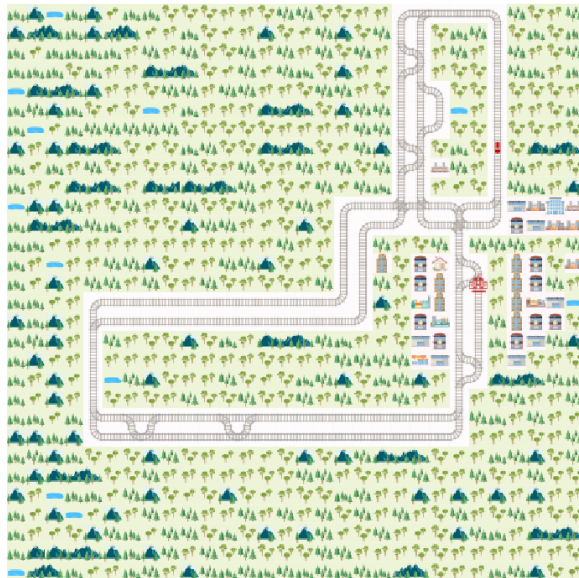


Figure 7.7: [Link to video](#)

Chapter 8

Custom Notations Added to the base environment

8.1 Description

Enhancements have been made to the Flatland environment to introduce advanced features and data integration:

- ****City Positions and Platforms****: Added functionality to specify precise city positions and include platforms within cities, enhancing the realism and complexity of railway networks.
- ****Customized City Connectivity****: Implemented options to manually define connections between cities, offering flexibility to create dense or sparse railway networks based on specific requirements.
- ****Custom Timetable Generator****: Developed a custom timetable generator to facilitate efficient scheduling of trains within the environment, optimizing resource allocation and transportation efficiency.
- ****Path Creation in Railway Environment****: Integrated algorithms for pathfinding within the railway environment, ensuring optimal routing and navigation for trains across the network.
- ****Geospatial Data Integration****: Scraped latitude and longitude coordinates for all railway stations and junctions in Tamil Nadu, embedding this geographical data into the custom Flatland Environment. This integration enhances geographical accuracy and supports location-based analyses and simulations.

These enhancements collectively contribute to a more sophisticated and realistic simulation environment for railway systems, enabling detailed analysis, optimization, and scenario testing in transportation planning and management.

8.2 Code Snippet

Below is a sample code snippet demonstrating the integration of city positions and platforms:

```

# Define cities with positions
cities = {
    'City-A': (10, 20),
    'City-B': (30, 40),
    'City-C': (15, 35)
}

# Add platforms to cities
for city, position in cities.items():
    add_platforms(city, num_platforms=2)

# Manually connect cities (dense network example)
connect_cities('City-A', 'City-B', connection_type='dense')
connect_cities('City-A', 'City-C', connection_type='sparse')

```

For further details and to explore the custom features and functionalities added, refer to the repository or documentation provided.

8.3 Observations

We want to optimize computation of observations only when it's needed, i.e. before making a decision. We update the dictionary `AGENT_REQUIRED_OBS` to tell the `ObservationBuilder` for which agent to compute obs. We compute observations only in these cases:

- Agent is entering switch (obs for last cell of current path): we need obs to evaluate which path to take next.
- Agent is exiting a switch (obs for new cell of new path): we compute the obs because we could immediately meet another switch (track section only has 1 cell), so we need the observation in buffer.
- Agent is about to finish: we compute obs to save the experience tuple.

Chapter 9

Future Work

Introduction

In this report, I outline a comprehensive approach to enhance navigation efficiency and decision-making in railway networks using Flatland environment. The methodology integrates advanced techniques such as Graph Neural Networks (GNNs), attention mechanisms, and reinforcement learning frameworks to achieve robust and explainable agent behaviors.

9.1 Utilizing Flatland Observations with Graph Neural Networks

To leverage the graph structure inherent in Flatland observations, we treat switches and stations as nodes and paths between them as edges. This approach allows us to model the railway network as a graph, where nodes represent decision points and edges denote possible transitions or paths.

9.2 Incorporating Attention Mechanisms into PPO (MAMBA) Algorithm

In enhancing the Proximal Policy Optimization (PPO) algorithm, specifically leveraging MAMBA (Multi-Agent Model-Based Reinforcement Learning with Attention), we incorporate attention mechanisms. Attention enables the agent to focus on relevant parts of the observation space, enhancing decision-making based on the importance of different graph nodes (switches and stations) and their relationships.

9.3 Explainability for Trajectories and State Importance

To provide transparency and interpretability in agent decisions, we introduce a method to analyze trajectories based on state importance. By identifying key features and nodes that contribute most significantly to action selection, we can explain why certain decisions are made in complex railway environments. This not only aids in

debugging and understanding agent behavior but also improves overall system reliability.

9.4 Framework Integration with Ray RLLib or CleanRL

To facilitate efficient utilization and training within the Flatland environment, we propose building a framework on top of existing tools such as Ray RLLib or CleanRL. This framework would streamline experimentation, hyperparameter tuning, and scalability, enabling rapid development and deployment of advanced navigation strategies.

9.5 Conclusion

By integrating Graph Neural Networks, attention mechanisms, explainability through state importance analysis, and leveraging robust frameworks like Ray RLLib or CleanRL, we aim to significantly enhance the performance and reliability of navigation systems in complex railway networks represented by the Flatland environment.

References

- [1] Mugurel-Ionut Andreica. Winning solution of the aicrowd sbb flatland challenge 2019-2020. AICrowd Flatland Challenge, 2020. <https://www.aicrowd.com/challenges/flatland-challenge>.
- [2] Sándor Kolumbán Anikó Kopácz, Ágnes Mester and Lehel Csato. Standardized feature extraction from pairwise conflicts applied to the train rescheduling problem. 2022. Emails: `aniko.kopacz@ubbcluj.ro`, `agnes.mester@ubbcluj.ro`, `sandor.kolumban@ubbcluj.ro`, `lehel.csato@ubbcluj.ro`.
- [3] Shao-Hung Chan, Zhe Chen, Dian-Lun Lin, Yue Zhang, Daniel Harabor, Tsung-Wei Huang, Sven Koenig, and Thomy Phan. Anytime multi-agent path finding using operation parallelism in large neighborhood search, 2024.
- [4] Zhe Chen, Daniel Harabor, Jiaoyang Li, and Peter J. Stuckey. Symmetry breaking for k-robust multi-agent path finding. *CoRR*, abs/2102.08689, 2021.
- [5] Zhe Chen, Jiaoyang Li, Daniel Harabor, and Peter J. Stuckey. Scalable rail planning and replanning with soft deadlines, 2024.
- [6] Vladimir Egorov and Alexei Shpilman. Scalable multi-agent model-based reinforcement learning. *JetBrains Research; HSE University*, 2022.
- [7] Florian Laurent, Manuel Schneider, Christian Scheller, Jeremy D. Watson, Jiaoyang Li, Zhe Chen, Yi Zheng, Shao-Hung Chan, Konstantin Makhnev, Oleg Svidchenko, Vladimir Egorov, Dmitry Ivanov, Aleksei Shpilman, Evgenija Spirovska, Oliver Tanevski, Aleksandar Nikov, Ramon Grunder, David Galevski, Jakov Mitrovski, Guillaume Sartoretti, Zhiyao Luo, Mehul Damani, Nilabha Bhattacharya, Shivam Agarwal, Adrian Egli, Erik Nygren, and Sharada P. Mohanty. Flatland competition 2020: MAPF and MARL for efficient train coordination on a grid world. *CoRR*, abs/2103.16511, 2021.
- [8] Sharada Prasanna Mohanty, Erik Nygren, Florian Laurent, Manuel Schneider, Christian Scheller, Nilabha Bhattacharya, Jeremy D. Watson, Adrian Egli, Christian Eichenberger, Christian Baumberger, Gereon Vienken, Irene Sturm, Guillaume Sartoretti, and Giacomo Spigler. Flatland-rl : Multi-agent reinforcement learning on trains. *CoRR*, abs/2012.05893, 2020.
- [9] Erik Nygren, Christian Eichenberger, and Emma Frejinger. Scope restriction for scalable real-time railway rescheduling: An exploratory study. 2022.
- [10] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.

- [11] Aravind Venugopal, Stephanie Milani, Fei Fang, and Balaraman Ravindran. Mabl: Bi-level latent-variable world model for sample-efficient multi-agent reinforcement learning, 2024.